ASRP: Adaptive Support-Based Response Prediction for Test-Time AI Capability Evaluation

Yuhan Chiu
National Taiwan University
Taipei, Taiwan
yhn.chiu@gmail.com

Coauthor name
Affiliation placeholder
Address placeholder
coauthor@example.com

Abstract

A single ability estimate can miss a subject's relative strengths across item types, while local adaptation from very few labels can overreact to individual outcomes. We present **ASRP (Adaptive Support-Based Response Prediction)**, which addresses this tension by shifting from public-data priors and content-based Rasch estimation at low label budgets to support-conditioned kernel residuals as evidence accumulates. A fixed budget-dependent ensemble controls this transition, shrunk topic corrections adjust systematic errors, and a streaming acquisition rule selects labels using covariance gain. On a public proxy covering 92 source groups and three seeds, ASRP achieves Brier ALC 0.1987 versus 0.2403 for Empirical Mean with identical adaptively acquired labels; switching ASRP from random to adaptive acquisition provides a smaller additional reduction of 0.0021.

1 Introduction

The Predictive AI Evaluation Competition (PAIEC) asks for an AI system's success probability on an individual item with at most 31 revealed labels per subject–benchmark pair. It requires both zero-label prediction and adaptation from sparse responses. Our target is query-level success, following the distinction from confidence in a particular generated answer made by Yang et al. [2026].

Shared structure and local evidence address different limitations. Item response theory models subject ability and item difficulty [Lord, 1980], while amortized evaluation predicts difficulty from content [Truong et al., 2025]. This shared structure supports cold-start prediction, but a scalar ability cannot express every relative strength across item types, and frozen estimates may transfer imperfectly to new benchmarks. Conversely, local corrections learned from one or two labels can overreact to individual outcomes. ASRP therefore combines prior-based estimation with regularized support-driven adaptation, increasing the latter's weight as evidence grows.

ASRP couples two prediction branches. A Rasch hybrid combines difficulty from frozen MiniLM representations [Wang et al., 2020], updated subject ability, and metadata-informed smoothing. A kernel probe learns item-dependent residuals around a smoothed subject baseline from revealed responses. **Budget-dependent mixing** controls their relative influence. Shrunk topic calibration corrects systematic public-data residuals, while covariance-based acquisition selects labels from the candidate stream. This integration is accompanied by released source selections, matched splits, predictions, and acquisition traces for reproducible evaluation.

The main measured improvement comes from prediction, with a smaller acquisition benefit. Across 92 public source groups and three seeds, ASRP reduces mean Brier ALC from 0.2403 to 0.1987 relative to Empirical Mean under identical Adaptive evidence, a reduction of 0.0416. Holding ASRP fixed, switching Random to Adaptive acquisition reduces ALC from 0.2008 to 0.1987 (0.0021). Component removals support the kernel at high budgets; six-draw Rasch ablations find a small zero-label benefit but essentially tied mean ALC. Ten separate official with-Rasch evaluations yield mean

Preprint.

ALC 0.1911, but expose a **cold-start transfer weakness**: zero-label Brier is 0.2640, above the 0.2500 of a constant 0.5 prediction. The later no-Rasch batch has worse mean ALC under different draws, motivating retention without establishing causality.

2 Competition Setting and Data

2.1 Prediction interface and metrics

At runtime the platform supplies a pair of dictionaries $x = (s, i)$ describing a subject and an item, together with a list of previously acquired binary observations. The predictor returns a finite probability in $[0, 1]$. Eligible subject–benchmark groups have at least 80 distinct items. The platform creates a fixed acquisition/evaluation split and evaluates the same evaluation items at the **six label budgets**

$$n \in \{0, 1, 3, 7, 15, 31\}.$$

Acquisition items are never scored and evaluation outcomes are never revealed. The optional acquisition hook receives the current candidate, acquired labels, and remaining-label context; the interface can also supply a current prediction when requested. ASRP uses the first three inputs.

For a budget n and a subject–benchmark group with N_{eval} evaluated responses, the Brier score is the mean squared error between the predicted probability p and the binary outcome y:

$$B_n = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} (p_j - y_j)^2.$$

The group-level Brier ALC is the normalized area on the logarithmic budget axis and is equivalent to

$$\text{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The official score averages these group-level ALC values equally across subject–benchmark groups. **Lower is better.** All reported local tables use this same weighting.

2.2 Public data and frozen artifacts

The predictor uses frozen artifacts trained and calibrated from the public `aims-foundations/measurement-db` release [AIMS Foundations, 2026]. Public responses provide subject priors and metadata statistics, while item content supplies semantic features. The local evaluation selects 92 source groups from a fixed inventory of 142 binary sources; Section 4 describes selection and splitting. Appendix B lists the evaluated sources. Upstream metadata and licenses remain authoritative.

The subject prior and metadata statistics are frozen public-response aggregates. Their runtime use and fallback rules are specified in Appendix A; reproducing the reported scores uses the released weights without refitting them. MiniLM is used as a pretrained encoder; the packaged encoder is `sentence-transformers/all-MiniLM-L6-v2`, based on MiniLM [Wang et al., 2020]. Its weights are **frozen at runtime**. The difficulty model, priors, kernel configuration, calibration statistics, and cluster centroids are prepared offline and bundled with the submission; kernel residual coefficients are fitted online from revealed labels. The current submission uses **no external LLM or web search API during scoring**. At runtime, the program uses only the supplied subject and item attributes, anonymous benchmark identifiers, and labels revealed within the allowed budget; it does not access hidden evaluation outcomes.

3 Proposed Method: ASRP

For a query subject s and item i in benchmark b, let $\mathcal{D}_b$ be the revealed support and $\mathcal{D}_s$ its rows for the current subject. Write $n = |\mathcal{D}_s|$ and $S_s = \sum_{r \in \mathcal{D}_s} y_r$. All predictions below concern this query; their arguments are suppressed when unambiguous. Figure 1 shows the computation. Appendix A gives the shared notation, numerical details, and corresponding code functions.

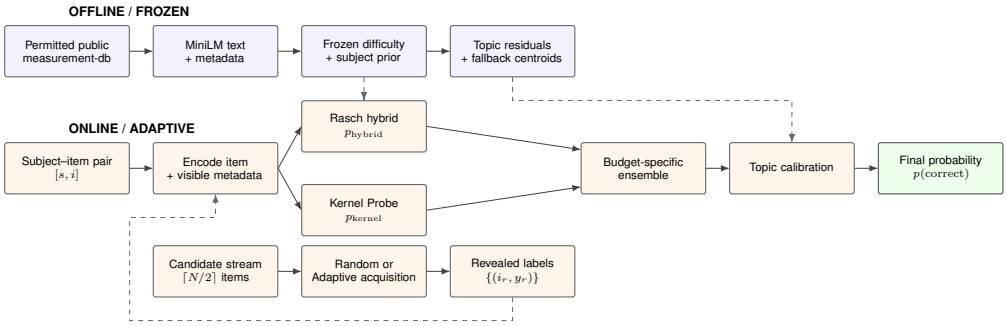


Figure 1: ASRP overview. Frozen public-data artifacts supply difficulty, prior, and calibration estimates. Revealed labels adapt the Rasch hybrid and kernel probe before budget-specific ensembling and topic correction. For a complete group of N items, the candidate pool contains $\lceil N/2 \rceil$ items. Random samples uniformly; Adaptive skips up to 16 warm-up candidates before covariance-gain selection. Both supply nested label prefixes.

3.1 Item representation and amortized difficulty

MiniLM encodes the normalized item text as $e_i \in \mathbb{R}^{384}$. Six deterministic numeric text features form $z_i \in \mathbb{R}^6$. A frozen feed-forward network standardizes their concatenation and predicts a clipped difficulty:

$$d_i = \text{clip}(f_\phi([e_i; z_i]), -2, 2)\,.$$

The network has dimensions 390–128–32–1 and ReLU hidden layers. This amortized estimate supplies a difficulty for an unseen item without fitting an item-specific parameter online. Appendix A.1 specifies text limits, pooling, and standardization.

3.2 Public prior and Rasch update

A frozen subject offset o_s, looked up by normalized model name, defines a prior mean and initial ability:

$$\mu_s = \text{clip}(0.5 + 0.5o_s, 0.05, 0.95), \qquad \theta_{0,s} = \text{logit}(\mu_s).$$

An unknown name has $o_s = 0$. With revealed labels, clipped Newton–Raphson iterations approximate the **maximum a posteriori (MAP) estimate** under a Bernoulli Rasch likelihood and Gaussian ability prior $\mathcal{N}(\theta_{0,s}, v_\theta)$, with $v_\theta = 2$. The resulting ability $\hat{\theta}_s$ gives

$$p_{\text{Rasch}} = \sigma(\hat{\theta}_s - d_i), \qquad \sigma(t) = (1 + e^{-t})^{-1}.$$

At $n = 0$, $\hat{\theta}_s = \theta_{0,s}$. Separately, a query-specific metadata prior $p_{\text{meta}}(s, i)$ combines frozen subject and visible-feature counts. The hybrid smooths observed successes toward this prior and blends the result with the Rasch probability:

$$p_{\text{smooth}} = \frac{S_s + \kappa p_{\text{meta}}(s,i)}{n + \kappa}, \qquad p_{\text{hybrid}} = (1 - \eta_n)p_{\text{smooth}} + \eta_n p_{\text{Rasch}}, \qquad \kappa = 4.$$

Here η_n is the frozen Rasch mixing weight. Appendix A.2 gives the prior lookup and Newton updates; Appendix A.4 lists the six mixing weights.

3.3 Support-conditioned kernel probe

The kernel branch adds an item-dependent correction to a conservative subject baseline. It encodes visible text, metadata, and interactor descriptions into semantic vectors $\mathbf{h}_i$ and lexical vectors $\boldsymbol{\ell}_i$. A locally normalized similarity kernel K is fitted using only the support features in $\mathcal{D}_b$; the query does not change this geometry.

For each supported subject s, let n_s and S_s be its revealed-label and success counts. A two-component Beta mixture has concentrations $(c_1, c_2) = (1, 8)$, prior masses $(\pi_1, \pi_2) = (0.3, 0.7)$, and posterior weights ω_{sk}. Its mean is

$$b_s = \sum_{k=1}^{2} \omega_{sk} \frac{c_k \mu_s + S_s}{c_k + n_s}.$$

Subject-centered kernel ridge regression, with penalty $\lambda_K = 0.3$, fits residual coefficients γ from the revealed labels. With query-to-support similarities $\mathbf{k}_i$ and subject mean fitted contribution $\bar{f}_s$, the prediction is

$$p_{\text{kernel}} = \text{clip}\big(b_s + \mathbf{k}_i^\top \gamma - \bar{f}_s, 0, 1\big)\,.$$

Appendix A.3 defines K, the Beta weight update, and the centered linear solve. If the current subject has no revealed labels, this branch returns μ_s. The implementation supports multiple subjects within a benchmark; this proxy uses one subject per group.

The ensemble gives the kernel weights $w_n = (0, 0, 0.2, 0.4, 1, 1)$ at budgets $(0, 1, 3, 7, 15, 31)$. Thus the Rasch hybrid supplies the cold-start prediction, while a larger support set allows the local residual to receive more weight. These are design choices encoded in the released configuration.

3.4 Topic calibration

A frozen topic correction $\Delta_n(i)$ adjusts the ensemble using the first matching topic entry in visible metadata. Without a match, the model uses the closest of eight frozen embedding centroids. Count-based shrinkage and caps limit the adjustment to 0.06 for explicit topics or 0.03 for clusters. Missing statistics give zero correction.

The budget-specific ensemble is followed by topic correction:

$$p_{\text{final}} = \text{clip}((1 - w_n)p_{\text{hybrid}} + w_n p_{\text{kernel}} + \Delta_n(i), 0, 1)\,.$$

Appendix A.4 gives the exact correction and lookup rules.

3.5 Adaptive acquisition

The `adaptive_4_warm` hook skips up to 16 early candidates to establish feature geometry, without revealing their labels. A reference set $\mathcal{P}$ deduplicates the last 64 seen candidates by visible text, metadata, and interactor features. For the current candidate $*$, a conditional covariance gain G_* is a feature-based uncertainty-reduction surrogate over $\mathcal{P}$. It depends on acquired-item features, but not on their outcome values, and does not directly estimate a change in Brier score. Normalizing and capping this gain, then raising it to power four, gives utility V_*.

For L labels and R candidates remaining, including the current candidate, the acquisition probability is

$$\pi_*^{\text{acq}} = \min\left(1, \frac{L}{R}\,\text{clip}\left(\frac{V_*}{\max(\overline{V}_{\mathcal{P}}, 10^{-12})}, 0.05, 20\right)\right), \qquad \overline{V}_{\mathcal{P}} = \frac{1}{|\mathcal{P}|}\sum_{j\in\mathcal{P}} V_j.$$

A deterministic uniform hash supplies the draw. When $R \le L$ and $L > 0$, every remaining candidate is acquired, overriding warm-up. Appendix A.5 defines G_j, V_j, and the boundary cases. The rule uses only acquired support and already-seen/current features; it never accesses future candidates or unseen outcomes.

4 Experimental Setup

We use paired public-data comparisons to separate prediction and acquisition effects, and report private-platform evaluations separately. Code, frozen artifacts, and exact reproduction commands are provided in Appendix D.

4.1 Public proxy protocol

Data and repeated splits. We select one eligible subject per source using a fixed seed and keep these subjects unchanged across experiments. Eligible groups contain 80–1,000 distinct items, all retained, yielding **92 source groups** and 44,741 items. The other 50 sources in the 142-source inventory have no qualifying group. The upper limit bounds local evaluation cost. We fix one binary outcome per item when repeated responses exist; official evaluation instead retains all recorded evaluation responses. For each seed (**42, 43, and 44**), items are split approximately equally into disjoint acquisition and evaluation pools. Candidate pools contain 45–500 items, and each seed has **22,357 evaluation outcomes**. The manifest records the data version, selected records, and splits.

Paired comparison. We evaluate **ASRP and Empirical Mean under both Random and Adaptive acquisition**. Random samples uniformly without replacement; Adaptive uses `adaptive_4_warm` from Section 3.5. Within each seed, both policies receive the same candidate order and evaluation pool. Both predictors receive identical acquired labels *within each policy*, while different policies may select different labels. A single acquisition trajectory targets 31 labels and supplies nested evidence at all six budgets; selection does not restart at each checkpoint. Every checkpoint evaluates the same held-out items.

Evaluation scope. Evaluation outcomes are never revealed online. However, public training, calibration, and proxy evaluation use the same data release, so this is not an independent test of all fitted components and local scores may be optimistic.

4.2 Baseline and score aggregation

We use the **Empirical Mean predictor** from the official PAIEC repository [Truong et al., 2026]:

$$p_{\mathrm{EM}} = \begin{cases} 0.5, & n = 0, \\ \frac{1}{n}\sum_{k=1}^{n} y_k, & n > 0. \end{cases}$$

It uses only the current subject–benchmark group's acquired labels, without item features or a learned prior. Random is its default acquisition policy; the Adaptive variant uses ASRP's selected labels.

We use the metrics and equal-group weighting in Section 2, then average the three seed-level scores equally. We report each seed's ALC and their sample standard deviation (SD), describing split variation. Source-level comparisons average paired ALC differences over the three seeds before counting improvements, with each selected source counted once.

4.3 Component ablations

We remove one prediction component at a time on the same 92 groups and three seeds, replaying the published Adaptive acquisition trajectories. **No Rasch** sets $\eta_n = 0$, retaining metadata smoothing and the kernel branch; **No Kernel Probe** sets $w_n = 0$, using the Rasch hybrid at every budget; **No topic calibration** sets $\Delta_n(i) = 0$. All other parameters and calibration statistics stay fixed, with no refitting or retuning. The full-model rerun is checked against every published Adaptive prediction within 10^{-7}. We additionally repeat the paired Rasch comparison on seeds **45, 46, and 47**. These six draws reuse the same selected sources and assess split sensitivity, not transfer to unseen sources.

4.4 Official formative feedback

We separately evaluate the same with-Rasch ASRP + `adaptive_4_warm` ZIP in 10 Codabench formative runs. Each run samples a private subset. We average group scores equally within each run, then report the mean and sample SD across runs using all six budgets. A later no-Rasch batch provides 16 completed platform scores; its one cancelled submission is excluded. Only total ALC is available for this batch. The two batches use different draws and dates and therefore form an **unpaired comparison**, distinct from the controlled public ablation. Appendix C records the submission IDs and available feedback.

5 Results and Analysis

5.1 Public proxy results

Table 1 reports the **four predictor–acquisition combinations**, averaged equally over groups and then over seeds 42, 43, and 44. ASRP + Adaptive has a mean Brier ALC of 0.1987, compared with 0.2435 for Empirical Mean + Random (an absolute reduction of 0.0447). Its aggregate Brier score is lower at all six budgets.

Predictor + acquisition	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
Empirical Mean + Random	0.2500	0.3488	0.2518	0.2086	0.1910	0.1842	0.2435
Empirical Mean + Adaptive	0.2500	0.3594	0.2366	0.2003	0.1883	0.1837	0.2403
ASRP + Random	0.2263	0.2166	0.2075	0.1947	0.1845	0.1755	0.2008
ASRP + Adaptive	0.2263	0.2161	0.2043	0.1912	0.1819	0.1738	0.1987

Table 1: Public proxy Brier scores averaged equally over 92 groups and seeds 42, 43, and 44. Lower is better. Evaluation pools are matched across policies; acquired labels are matched across predictors within each policy.

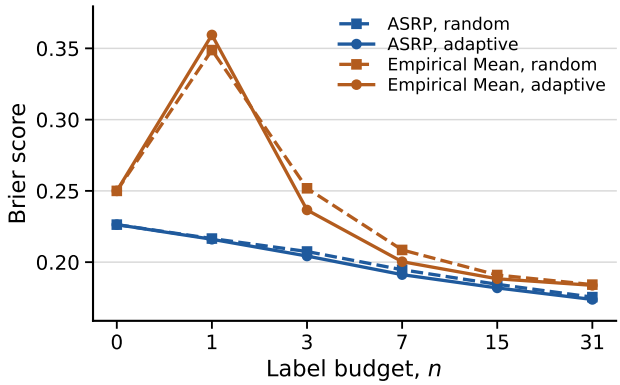


Figure 2: Mean public-proxy Brier curves for the four predictor–acquisition combinations on the official $\log_2(1+n)$ budget axis. Solid lines denote Adaptive acquisition and dashed lines Random; values match Table 1.

Predictor + acquisition	Seed 42	Seed 43	Seed 44	SD
Empirical Mean + Random	0.2467	0.2401	0.2435	0.0033
Empirical Mean + Adaptive	0.2386	0.2427	0.2396	0.0021
ASRP + Random	0.2020	0.1984	0.2020	0.0021
ASRP + Adaptive	0.1983	0.1983	0.1995	0.0007

Table 2: Group-averaged Brier ALC for each split/acquisition seed. SD is the sample standard deviation over the three seed scores. Lower ALC is better.

5.2 Prediction and acquisition effects

Predictor effect with acquisition held fixed. Relative to Empirical Mean with matched labels, ASRP reduces ALC by 0.0426 under Random and 0.0416 under Adaptive acquisition. Empirical Mean + Random rises from 0.2500 at zero labels to 0.3488 at one label: a single outcome makes its prediction either zero or one. ASRP smooths early evidence using public priors. This comparison measures the complete predictor, including its frozen public-data information; component removals are examined in Section 5.3.

Acquisition effect with the predictor held fixed. For ASRP, Adaptive lowers mean ALC by 0.0021 relative to Random, and has lower ALC in all three seeds. For Empirical Mean, Adaptive lowers mean ALC by 0.0031 relative to Random, and has lower ALC in 2 of the three seeds. The closest ASRP pair differs by only -0.00005 (Adaptive minus Random, seed 43). Table 2 shows the individual seed results and their sample standard deviation; three seeds summarize sensitivity to these draws rather than establishing universal superiority.

Variation across sources. Averaging the paired ALC differences over seeds within each source, ASRP beats Empirical Mean on 82 of 92 sources under Adaptive acquisition and loses on 10. For

ASRP, Adaptive improves on Random in 60 sources, worsens in 31, and ties in 1 (tolerance 10^{-12}). These counts describe the selected subject per source.

5.3 Component contributions

Variant	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC	ΔALC
Full ASRP	0.2263	0.2161	0.2043	0.1912	0.1819	0.1738	0.1987	–
No Rasch	0.2277	0.2154	0.2035	0.1912	0.1819	0.1738	0.1985	-0.0002
No Kernel Probe	0.2263	0.2161	0.2052	0.1936	0.1865	0.1829	0.2012	$+0.0025$
No topic calibration	0.2285	0.2176	0.2049	0.1918	0.1820	0.1739	0.1995	$+0.0008$

Table 3: Component removals with fixed Adaptive evidence and remaining parameters, averaged over 92 groups and three seeds. Positive ΔALC (variant minus full ASRP) indicates worse performance.

Kernel adaptation contributes at higher budgets. Removing the kernel branch increases ALC from 0.1987 to 0.2012, the largest increase among the three removals (0.0025). At budget 31, Brier worsens from 0.1738 to 0.1829. This supports the kernel branch at higher budgets; its zero weight leaves budgets zero and one unchanged.

Calibration and Rasch have smaller, different effects. Removing topic calibration increases ALC to 0.1995 ($+0.0008$); its largest Brier benefit is at budget zero. Removing Rasch slightly lowers ALC to 0.1985 (-0.0002), although zero-label Brier worsens from 0.2263 to 0.2277. The small ALC change varies in sign across seeds. Rasch helps initialization but not mean ALC here. Metadata smoothing and the kernel baseline remain, so the broader improvement over Empirical Mean cannot be assigned to Rasch alone.

Paired variant	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
With Rasch	0.22583	0.21652	0.20430	0.19132	0.18182	0.17380	0.19875
Without Rasch	0.22696	0.21619	0.20379	0.19144	0.18182	0.17380	0.19872

Table 4: Rasch ablation extended to seeds 42–47 on the same 92 source groups. Each pair shares acquired labels and evaluation items; remaining parameters are fixed.

Why retain Rasch? Across six matched draws, retaining Rasch lowers zero-label Brier from 0.22696 to 0.22583, with the same direction in all 6 seeds (Table 4). Overall ALC is essentially tied: 0.19875 with Rasch versus 0.19872 without it; retention improves ALC in only 2 of six seeds. Budgets 15 and 31 are identical because the hybrid has zero weight. **The controlled evidence supports a small initialization benefit, not an aggregate ALC advantage.** We retain the original limited-weight Rasch term to preserve this cold-start contribution; the less favorable subsequent no-Rasch official batch adds a practical reason to withdraw the simplification (Section 5.4). All six draws reuse the same sources, and the official batches are unpaired, so neither establishes a general transfer benefit.

5.4 Official formative results

Across 10 official formative evaluations of the same ASRP submission with adaptive acquisition, **mean Brier ALC is** 0.1911 (sample SD 0.0111; Table 5). These official samples are separate from the local public-proxy comparisons.

ASRP + Adaptive	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
Mean	0.2640	0.2276	0.1924	0.1686	0.1593	0.1511	0.1911
SD	0.0372	0.0236	0.0145	0.0185	0.0158	0.0140	0.0111

Table 5: Official Brier scores and ALC for ASRP with adaptive acquisition: mean and sample SD over 10 evaluations, with equal group weighting within each evaluation. Lower is better.

Official batch	Runs	Mean ALC	SD
With Rasch	10	0.19107	0.01105
Without Rasch	16	0.20893	0.02813

Table 6: Unpaired official batches. The with-Rasch batch was evaluated on October 27 and the no-Rasch batch on October 30, 2026. SD describes variation across completed runs.

The 16 completed no-Rasch submissions in the later platform screenshot have mean ALC 0.20893 (SD 0.02813), compared with 0.19107 (SD 0.01105) for the 10 with-Rasch evaluations. The cancelled submission is excluded. These batches use different random draws and dates; only total ALC is available for the no-Rasch batch. **This unpaired comparison** motivates retaining the prior submission, but cannot identify Rasch as the cause or locate the difference by label budget.

6 Limitations and Discussion

Cold-start transfer. Frozen public priors, difficulty estimates, and topic corrections may transfer poorly before labels are available. Across 10 official evaluations, zero-label Brier is 0.2640, above the 0.2500 of a constant 0.5 prediction. Revealed evidence reduces it to 0.1511 at budget 31. The public ablations do not identify which initialization component causes the private-data errors; accurate cold-start prediction remains a limitation.

Evidence for model selection. The public proxy shares its data release with offline fitting and calibration, so local gains may be optimistic. Six seeds on the same sources do not constitute six independent generalization tests. The official batches also differ in sampled subjects and items; their ALC gap cannot establish a causal Rasch benefit. Without per-budget feedback for the no-Rasch batch, we cannot locate its regression at cold start. Retaining Rasch is a conservative model-selection decision, not evidence that it is indispensable.

7 Conclusion

ASRP balances public-data initialization with support-driven adaptation as labels accumulate. Paired experiments show a larger predictor benefit than acquisition benefit, with component removals supporting kernel adaptation at higher budgets and topic calibration. We retain Rasch for its small local initialization benefit and because its removal has no established overall advantage; the later official batch reinforces caution about simplification. Accurate cold-start transfer remains a challenge. Released artifacts and matched splits make these comparisons reproducible.

A Mathematical and Implementation Details

These equations specify inference with the released frozen artifacts at the six competition budgets. They use the same symbols as Section 3. The reproduction command evaluates these artifacts without repeating their original offline fitting. Throughout, $\mathrm{clip}(x, l, u) = \min(u, \max(l, x))$, applied coordinatewise to vectors, and $\mathrm{norm}(x) = x/\max(\|x\|_2, 10^{-12})$.

Shared notation. A support row r contains a subject s_r, item i_r, and binary outcome y_r. The row collection $\mathcal{D}_b$ contains all revealed responses for benchmark b and has size M. Its subject-specific subset $\mathcal{D}_s$ has n_s rows and S_s successes. For the current query subject, the budget is $n = n_s \le 31$. In this proxy $M = n$; the formulas below also describe the implementation's multi-subject kernel. Runtime subject identity uses the complete subject dictionary, whereas frozen subject-prior lookup uses `normalized_name`.

We distinguish η_n (Rasch mixing weight), w_n (kernel mixing weight), and λ_K (ridge penalty). Beta component index $k \in \{1, 2\}$ has concentration c_k, prior mass π_k, and posterior mass ω_{sk}. Acquisition uses gain G_j, utility V_j, probability π_*^{acq}, and uniform variate ξ; none denotes an embedding coordinate.

A.1 Feature construction for Section 3.1

The Rasch encoder normalizes whitespace, takes the first 6,000 characters of item content, and tokenizes to at most 256 tokens. Masked mean pooling followed by L_2 normalization gives $e_i \in \mathbb{R}^{384}$. The six coordinates of z_i are $\log(1+\text{text length})/10$, digit fraction, non-ASCII fraction, and indicators for code, mathematical notation, and a question mark. Fractions and the question-mark indicator use the first 2,000 characters; code/math markers use the first 6,000. Text length is measured before these truncations.

For stored feature mean $\bar{\mathbf{x}}$ and scale $\boldsymbol{\sigma}_x$, the difficulty network receives $\mathrm{clip}(([e_i; z_i] - \bar{\mathbf{x}}) \oslash \boldsymbol{\sigma}_x, -10, 10)$, where $\oslash$ denotes coordinatewise division. Frozen weights in `difficulty_model.npz` implement the 390–128–32–1 map; the final scalar is clipped to $[-2, 2]$ by `_predict_hybrid`, after `_difficulty` evaluates the network.

The kernel encoder uses a separate text construction containing item content, metadata, and interactor descriptions. It pools up to four evenly spaced windows of 254 content tokens, each with special tokens, normalizes each pooled vector, and normalizes their sum to obtain $\mathbf{h}_i \in \mathbb{R}^{384}$. The lexical vector $\boldsymbol{\ell}_i \in \mathbb{R}^{8192}$ uses signed CRC32 hashing of lowercased tokens and adjacent token bigrams, weights $1 + \log(\text{count})$, and L_2 normalization. Only these semantic and lexical coordinates enter the kernel. See `ml_features.py`, `window_features.py`, and `evidence_features.py` under `src/asrp/`.

A.2 Priors and ability update for Section 3.2

Two distinct frozen priors. The scalar o_s comes from `binary_subject_prior.json`. It defines μ_s and $\theta_{0,s}$ exactly as in Section 3.2. The metadata prior $p_{\mathrm{meta}}(s, i)$ instead uses the success/count tables in `feature_prior.json`; it corresponds to `_feature_prior` and the runtime variable `p_source` in `src/asrp/model.py`. The released source-prior weight is one, prior shift is zero, and metadata-match adaptation weight is zero.

Let (A_0, N_0) be global public successes and counts, and (A_s, N_s) the subject totals. The frozen rates are

$$q_0 = \frac{A_0}{\max(N_0, 1)}, \qquad q_s = \frac{A_s + 8q_0}{N_s + 8}.$$

Unknown subjects use global totals. Their offset o_s is separately zero, so $\theta_{0,s} = 0$, but $p_{\mathrm{meta}}(s, i)$ need not equal 0.5. All released global totals have positive counts.

For each token t in the query metadata with stored global counts, let $q_t = A_t/\max(N_t, 1)$. When subject–token counts exist,

$$q_{st} = \frac{A_{st} + 8q_t}{N_{st} + 8}, \qquad \rho_{st} = \frac{\min(N_{st}, 80)}{\min(N_{st}, 80) + 12}.$$

Otherwise $q_{st} = q_t$ and $\rho_{st} = \min(N_t, 300)/[\min(N_t, 300) + 100]$. For the set $\mathcal{T}(i)$ of matching tokens,

$$p_{\text{meta}}(s, i) = \text{clip}\left(\frac{2q_s + \sum_{t\in\mathcal{T}(i)} \rho_{st} q_{st}}{2 + \sum_{t\in\mathcal{T}(i)} \rho_{st}}, 0.03, 0.97\right).$$

An empty token set gives clipped q_s. Token parsing, deduplication, and stop words follow `_feature_tokens`. The packaged artifact is present; if it were absent, the runtime fallback would be μ_s.

MAP objective and numerical update. Using only $\mathcal{D}_s$, the penalized Bernoulli log-likelihood is

$$\mathcal{L}_s(\theta) = \sum_{r\in\mathcal{D}_s} [y_r \log \sigma(\theta - d_{i_r}) + (1 - y_r) \log(1 - \sigma(\theta - d_{i_r}))] - \frac{(\theta - \theta_{0,s})^2}{2v_\theta}, \qquad v_\theta = 2.$$

Starting at $\theta_{0,s}$, `update_ability` computes

$$\begin{aligned} p_r(\theta) &= \sigma(\text{clip}(\theta - d_{i_r}, -40, 40)), \\ J_s(\theta) &= \sum_{r\in\mathcal{D}_s} (y_r - p_r(\theta)) - (\theta - \theta_{0,s})/v_\theta, \\ \mathcal{I}_s(\theta) &= \sum_{r\in\mathcal{D}_s} p_r(\theta)(1 - p_r(\theta)) + 1/v_\theta, \\ \delta_\theta &= \text{clip}\big(J_s(\theta)/\max(\mathcal{I}_s(\theta), 10^{-6}), -2, 2\big), \qquad \theta \leftarrow \theta + \delta_\theta. \end{aligned}$$

Here J_s is the gradient and $\mathcal{I}_s$ the positive negative-Hessian magnitude, not the Hessian itself. The routine stops after 30 iterations or when $|\delta_\theta| < 10^{-7}$, then clips $\hat{\theta}_s$ to $[-12, 12]$. With no labels it returns $\theta_{0,s}$ directly. The runtime sigmoid also clips its logit to $[-40, 40]$ when computing p_{Rasch}. The hybrid uses at most the first 31 matching labels in acquisition order.

A.3 Kernel calculations for Section 3.3

Support-only geometry and bandwidth. For $M > 1$ rows in $\mathcal{D}_b$, let $\bar{\mathbf{h}}$ and v_j^{feat} be the coordinatewise semantic mean and population variance (divisor M). Define semantic scales t_j and lexical weights ν_ℓ by

$$t_j = \left[v_j^{\text{feat}} + \max(\text{median}_j\, v_j^{\text{feat}}, 10^{-7})\right]^{-1/2}, \qquad \nu_\ell = \log\frac{1 + M}{1 + f_\ell} + 0.1,$$

where $f_\ell = \sum_{r\in\mathcal{D}_b} \mathbf{1}[|\ell_{i_r,\ell}| > 10^{-10}]$. Both support and query use

$$\widetilde{\mathbf{h}}_i = \text{norm}((\mathbf{h}_i - \bar{\mathbf{h}}) \odot \mathbf{t}), \qquad \widetilde{\boldsymbol{\ell}}_i = \text{norm}(\boldsymbol{\ell}_i \odot \boldsymbol{\nu}).$$

For $M = 1$, the mean is zero and both weight vectors are all ones. No query feature enters these fitted statistics.

For support rows r, t, define

$$Q_{rt} = \tfrac{1}{2}\left(\widetilde{\mathbf{h}}_{i_r}^\top \widetilde{\mathbf{h}}_{i_t} + \widetilde{\boldsymbol{\ell}}_{i_r}^\top \widetilde{\boldsymbol{\ell}}_{i_t}\right), \qquad K_{rt} = \exp\left(\frac{\min(Q_{rt} - 1, 0)}{h_K}\right).$$

Take distances $\max(1 - Q_{rt}, 0)$ over unordered off-diagonal pairs, retain those above 10^{-8}, and let d_{med} be their median (or 0.1 if none remain). The bandwidth is $h_K = 4\max(d_{\text{med}}, 10^{-5})$. The vector $\mathbf{k}_i \in \mathbb{R}^M$ uses this same transformation and bandwidth between query i and each support row. These operations correspond to `fit_geometry`, `fit_evidence`, and `predict_fitted` in `src/asrp/project_core.py`.

Beta mixture. For component $k \in \{1, 2\}$, let $\alpha_{sk} = \mu_s c_k$ and $\beta_{sk} = (1 - \mu_s)c_k$, with $(c_1, c_2) = (1, 8)$. The normalized posterior masses are

$$\widetilde{\omega}_{sk} = \pi_k \frac{\text{B}(\alpha_{sk} + S_s, \beta_{sk} + n_s - S_s)}{\text{B}(\alpha_{sk}, \beta_{sk})}, \qquad \omega_{sk} = \frac{\widetilde{\omega}_{sk}}{\sum_{k'=1}^{2} \widetilde{\omega}_{sk'}}, \qquad (\pi_1, \pi_2) = (0.3, 0.7).$$

Here B is the beta function. The implementation evaluates log-beta functions and normalizes by log-sum-exp. These weights give b_s in Section 3.3. If the current subject is absent from the support, `kernel_probe.py` returns μ_s without fitting a residual for that subject.

Centered ridge fit. Let $Z \in \{0,1\}^{M\times J}$ be the membership matrix for the J subjects present in the support, and $D = \mathrm{diag}(n_1, \ldots, n_J)$. Every diagonal count is positive. Let I_M be the $M \times M$ identity and $\mathbf{y} = (y_r)_{r\in\mathcal{D}_b} \in \mathbb{R}^M$. Define

$$H = I_M - ZD^{-1}Z^\top, \qquad K_c = HKH.$$

$$\gamma = H\left[\tfrac{1}{2}(K_c + K_c^\top) + \lambda_K I_M\right]^{-1} H\mathbf{y}, \qquad \lambda_K = 0.3.$$

The target $H\mathbf{y}$ subtracts each subject's empirical mean; the Beta mean b_s enters the final prediction separately. The code evaluates the inverse expression with a linear solve. Symmetrization matches the floating-point implementation; in exact arithmetic K_c is symmetric. The fitted subject contribution is

$$\bar{f}_s = \frac{1}{n_s}\sum_{r\in\mathcal{D}_s} (K\gamma)_r.$$

Substituting b_s, γ, $\mathbf{k}_i$, and $\bar{f}_s$ into Section 3.3 gives the implemented kernel prediction. At $n_s = 0$ this residual expression is bypassed; at $n = 0, 1$ the final ensemble weight w_n is zero.

A.4 Mixing and calibration for Section 3.4

The six released mixing weights are

n	0	1	3	7	15	31
η_n	0.2875	0.3000	0.2875	0.2750	0.2250	0.2375
w_n	0	0	0.2	0.4	1	1

Here η_n maps to `runtime_config.json:lambdas`; w_n is the active `PROBE_WEIGHTS` in `src/asrp/model.py`; and λ_K maps to `probe_config.json:ridge`. They are distinct parameters. The formulas in this report concern the official checkpoint counts; defensive interpolation/fallback behavior at other counts is not used in the experiment.

Let $t = t(i)$ be the first explicit topic key–value entry, in visible metadata order, present in the frozen topic table. If none matches, assign the Rasch embedding e_i to the closest of eight frozen centroids by cosine similarity and use its cluster entry. Write C_t^{cal} for that entry's frozen count and $r_{t,n}$ for its signed correction at budget n. Then

$$\Delta_n(i) = \mathrm{clip}\left(\frac{C_t^{\mathrm{cal}}}{C_t^{\mathrm{cal}} + a_t^{\mathrm{cal}}}\, r_{t,n}, -d_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}\right), \qquad (a_t^{\mathrm{cal}}, d_t^{\mathrm{cal}}) = \begin{cases} (40, 0.06), & \text{explicit topic,} \\ (200, 0.03), & \text{cluster.} \end{cases}$$

If no entry exists, $\Delta_n(i) = 0$. Cluster cosine normalization floors norms at 10^{-8} and resolves ties by the first maximum. At the six checkpoints, the residual array index is the corresponding budget index.

The correction $\Delta_n(i)$ is added after mixing the hybrid and kernel predictions with weight w_n. A final clip to $[0, 1]$ completes `predict`.

A.5 Covariance calculation for Section 3.5

The acquisition kernel is denoted K^{acq} to distinguish it from the prediction kernel K. Its geometry is fitted to the concatenation of acquired support $\mathcal{D}_b$, the reference list $\mathcal{P}$, and the current candidate $*$. The same visible item may appear in both support and reference positions; these occurrences remain in the concatenation. Only $\mathcal{P}$ itself is deduplicated. Thus acquisition may use seen but unlabeled features, whereas prediction geometry uses only acquired support features.

Use the feature normalization and RBF rule in Appendix A.3, but compute the median-distance bandwidth from pairs in $\mathcal{P}$ only. Let $\mathcal{U}$ concatenate $\mathcal{P}$ and $*$. For the current subject s, augment the kernel blocks as

$$A_{\mathcal{D}\mathcal{D}}[r,t] = K^{\mathrm{acq}}_{\mathcal{D}\mathcal{D}}[r,t] + 0.25\,\mathbf{1}[s_r = s_t],$$
$$A_{\mathcal{U}\mathcal{D}}[j,r] = K^{\mathrm{acq}}_{\mathcal{U}\mathcal{D}}[j,r] + 0.25\,\mathbf{1}[s_r = s],$$
$$A_{\mathcal{U}\mathcal{U}} = K^{\mathrm{acq}}_{\mathcal{U}\mathcal{U}} + 0.25\mathbf{1}\mathbf{1}^\top,$$

where block subscript $\mathcal{D}$ abbreviates $\mathcal{D}_b$. Conditional covariance and gain are

$$\Sigma = A_{\mathcal{U}\mathcal{U}} - A_{\mathcal{U}\mathcal{D}}(A_{\mathcal{D}\mathcal{D}} + I_M)^{-1}A_{\mathcal{U}\mathcal{D}}^\top, \qquad G_j = \max\left(\frac{|\mathcal{P}|^{-1}\sum_{r\in\mathcal{P}}\Sigma_{rj}^2}{1 + \max(\Sigma_{jj}, 0)}, 10^{-12}\right).$$

The identity term implements unit observation noise; without support, the subtraction is omitted. Define

$$\overline{G}_{\mathcal{P}} = |\mathcal{P}|^{-1} \sum_{r \in \mathcal{P}} G_r, \qquad V_j = \left[\min\left(\frac{G_j}{\max(\overline{G}_{\mathcal{P}}, 10^{-12})}, 20 \right) \right]^4 .$$

These utilities enter π_*^{acq} in Section 3.5. The power four is applied before dividing by mean reference utility. Gain depends on support membership and features; the numerical values of the revealed outcomes do not enter this covariance calculation.

Acquisition rules and reproducibility. Let A be the initial candidate count and $B_{\max} = 31$ the total label allowance in this experiment. Warm-up skips the first $W = \min(16, \max(0, A - B_{\max}))$ candidates. Checks occur in this order: reject if $L \leq 0$; acquire if $R \leq L$; otherwise skip during warm-up. After warm-up, an empty reference list falls back to probability L/R; otherwise use π_*^{acq}.

To ensure reproducibility, we generate a deterministic pseudorandom value $\xi \in [0, 1)$ from the current context and input using SHA256. We acquire the label when $\xi < \pi_*^{\mathrm{acq}}$, using L/R as the fallback acquisition probability. The reference list uses the last 64 candidates before the current one, deduplicated by the hash of visible item text/metadata/interactor content. Skipped outcomes are never supplied. See `AdaptiveGainPolicy.choose` in `src/asrp/adaptive_gain_policy.py`.

B Public source inventory

The following 92 public binary source identifiers occur in the paired three-seed evaluation. They identify the evaluation sources; the benchmark inventory and upstream licenses remain authoritative.

```
afrieval
agentic_review_perturbation
algotune
alignment_faking
appworld
averimatec
babilong
beavertails
benger
bird_sql
braveguard
bridging_gap
cellverse
critic_discernment_game
crow
devbench
editbench
edu_circuit_hw
egoschema
elicitation_game
engdesign
enginemt_qa
exploitgym
faithcot
fewshot_ttt_bbh
financebench
finegrain_t2i
hallusionbench
harmbench
healthadminbench
helm_bbq
helm_thaiexam
hle
igakuqa119
ikp
jailbreakbench
kernelbench
legal_rag_bench
llmail_inject
llms_lean_formalization
mathconstraint
mathvista_mini
medhelm_ehr_sql
medhelm_head_qa
medhelm_med_mcqa
medhelm_med_qa
medhelm_medbullets
```

```
medhelm_medcalc_bench
medhelm_medec
medhelm_medhallu
medhelm_pubmed_qa
medhelm_race_based_med
mj_bench
mmbench_v11
monkey_power_laws
morphkv
mtbbench
multi_swebench
naep_llm_students
naturebench
nester
nyu_ctf_bench
openbiorq
os_harm
perfcodebench
planbench
proofaug
pxplore
real_webagents
realtimeqa
reliancescope
researchcodebench
revolve
risebench
robust_reasoning_benchmark
safeagentbench
sib200
software_world_models
sustainable_food
swe_smith
swebench_live
swepolybench
tau2_bench
taubench
truthfulqa_mc
tulu_human_eval
tumlu
visual_riddles
visualwebarena
vl_rewardbench
workarena
workbench_revisited
```

C Official feedback provenance

We evaluated the same ASRP + `adaptive_4_warm` ZIP in 10 official formative evaluations. Table 7 lists the submission IDs and corresponding results.

Submission ID	Groups	Eval. items	Brier ALC
1012098	8	465	0.185821
1012118	9	475	0.193108
1012128	9	492	0.200457
1012139	9	492	0.173472
1012173	9	493	0.192103
1012180	9	493	0.173130
1012205	9	497	0.190750
1012212	8	456	0.205027
1013248	9	491	0.203112
1013255	9	494	0.193747

Table 7: Official formative evaluations identified by Codabench submission ID. Brier ALC is reconstructed from the rounded group feedback and may differ from the platform display by 10^{-6}. Evaluation-item counts do not determine group weights.

Submission ID	Brier ALC	Submission ID	Brier ALC
1020254	0.189257	1020290	0.240898
1020256	0.175680	1020330	0.168412
1020257	0.181208	1020332	0.221960
1020258	0.265550	1020339	0.179669
1020259	0.224247	1020340	0.213506
1020271	0.250078	1020343	0.219614
1020275	0.184611	1020346	0.207015
1020279	0.205590	1020350	0.215615

Table 8: Completed no-Rasch submissions transcribed from the October 30 platform screenshot. Cancelled submission 1020351 is excluded. Only total ALC is available.

The earlier ten evaluations were confirmed to use the same with-Rasch ZIP. The local inference files are restored to the pre-removal revision. Historical platform ZIP hashes are unavailable; the no-Rasch batch association follows our submission record and screenshot, rather than a hash recovered from Codabench.

D Reproducibility and Disclosure

Code, frozen weights, evaluation manifests, predictions, acquisition traces, and report source are available at `https://github.com/Hubert0916/arsp`. After accepting the measurement-db access conditions, run from the repository root:

```
uv sync --locked
uv run hf auth login
uv run --locked python tools/reproduce.py
```

The pipeline reconstructs the pinned public-data splits, evaluates all four combinations across three seeds on CPU, and checks predictions within 10^{-7} and acquired labels exactly against the published results. A full rerun of the restored package reproduced both output files byte-for-byte. Reproduction uses the released frozen artifacts; original offline fitting scripts are not included.

For the fixed-evidence component ablations in Section 5.3, run `tools/evaluate_ablation.py` after the main pipeline has prepared the data. Released compressed predictions can be audited and the ablation table regenerated with `tools/update_ablation_report.py`.

The additional Rasch comparisons use `tools/confirm_rasch_removal.py`; their pinned implementation and seeds are recorded in `experiments/rasch_confirmation.json`. Run `tools/update_rasch_report.py` to regenerate the six-seed comparison from both sets of paired predictions.

Selection capacity. Across the full experiment, 273 of 276 Adaptive trajectories execute at least one discretionary decision. The 3 trajectories without discretionary choices belong to one 90-item group: 14 warm-up skips leave exactly 31 candidates, all mandatory.

Released official aggregates and scripts regenerate the official result tables; private platform evaluations cannot be rerun locally. File descriptions and build instructions are provided in the repository README.

Competition-specific artifacts use permitted measurement-db data and the pretrained MiniLM encoder. Codex assisted development and report preparation; no external LLM API is used during scoring.

References

Sin-Han Yang, Cheng-Kuang Wu, Chieh-Yen Lin, Yun-Nung Chen, Hung-yi Lee, and Shao-Hua Sun. On calibration of large language models: From response to capability. *arXiv preprint arXiv:2602.13540*, 2026. URL `https://arxiv.org/abs/2602.13540`.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025. URL https://arxiv.org/abs/2503.13335.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2002.10957.

AIMS Foundations. measurement-db: Public benchmark inventory and response data. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/aims-foundations/measurement-db.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL https://github.com/aims-foundations/paiec_baseline.